3. Крайни автомати. Регулярни езици. Теорема на Клини.

$M = (K, \Sigma, \delta, s, F)$

Дефиниция: Краен детерминиран автомат е наредената петорка, където:

K

е крайно множество от състояния

Σ

е крайна азбука

$s \in K$

е началното състояние

$F \subseteq K$

е множеството от крайни състояния

$K \times \Sigma$

е функцията на преходите. Това е функция $\delta : \rightarrow$

$\delta(q, a) \in K$

Функцията на преходите определя това как се сменя текущото състояние според това каква е текущата буква в думата. Например, ако се намираме в състояние и текущата буква е , тогава следващото състояние в което трябва да се преместим е .

$(q_2, ababab)$

Дефиниция: Текуща конфигурация на автомата ще наричаме текущото състояние, в което се намираме и остатъка от думата, който не е прочетен. Както казахме по-рано това по еднозначен начин дефинира текущото състояние на изпълнение на автомата, тъй като по никакъв начин изпълнението не зависи от буквите, които са вече прочетени. Ще записваме конфигурациите с наредени двойки: , който означават състояние и остатъка от думата.

$e(q, e)) = q' (q', w')$

Дефиниция: Ако и са две конфигурации на M, тогава е изпълнено тогава и само тогава когато за някои символ и . Ако се намираме в конфигурацията , това означава, че сме прочели цялата дума и работата на автомата е приключила (тук използваме като означение на празната дума).

$(q, w)) \quad {}^{*}_{M} (q', w')$

Ще означаваме <u>рефлексивната и транзитивно затворена релация</u> на със , което означава, че ако , то тогава е достижимо от след последователност от преходи или дори без нито един преход. Така можем да дефинираме и кога една дума се разпознава от автомат M.

$L(M) = \{w : w \in \Sigma^*, (s,w) \bullet \overset{*}{}_{M} (q,e), q \in F\}$

Дефиниция: една дума <u>се разпознава</u> от автомат M, тогава и само тогава когато съществува състояние , такова че . Така казваме, че думата е от езика, който автомата разпознава, т.е. , където <u>дефинираме</u>.

$M = (K, \Sigma, \Delta, s, F)$

Дефиниция: <u>Недетерминиран краен автомат</u> е наредената петорка ,

където:

K

е крайно множество от състояния

Σ

е крайна азбука

$s \in K$

е началното състояние

$F \subseteq K$

е множеството от крайни състояния

$K \times (\Sigma \cup \{e\})$

е множество от преходите. : → (2 ?)

$\bullet_{M} p) \in \Delta$

Всяка една наредена тройка се нарича преход в M от в при входна буква . Възможно е да имаме преходи от вида , при които от преминаваме в без да четем буква от входа. Конфигурация дефинираме по същият начин както при детерминираните автомати. Релацията , се дефинира по аналогичен начин.

$L(M_1) = L(M_2)$

Дефиниция: Два автомата и са <u>еквивалентни</u>

Вече можем да въведем и следната теорема:

Теорема: <u>За всеки недетерминиран автомат, съществува еквивалентен детерминиран автомат.</u>

<u>Доказателство</u>:

[redacted]

Нека е КНА. Ще построим КДА еквивалентен на .Идеята е да си представим, че на всяка стъпка не се намираме в едно състояние, а в множество от състояния, което е множеството на всички състояния, в които можем да се намираме в момента за прочетената дума. Така ако имаме 5 състояния и за текущо прочетената дума можем да се намираме в състоянията и , можем да смятаме, че се намираме в състояние . Ако следващият символ от входа премества от в или , от в и от в , тогава следващото състояние за тази буква ще е .

[redacted]

Използвайки тази идея започваме конструирането. Логично, състоянията на ще бъдат . Крайните състояния на новата машина ще бъдат всички тези състояния, които съдържат в себе си крайно състояние на недетерминирания автомат. Функцията на преход е малко по сложна, поради това че при всеки един преход в недетерминираната машина е възможно да извършим и произволен брой -преходи. За да формализираме това, ще въведем следната дефиниция:

[redacted]

Дефиниция: За дадено състояние , нека , т.е. това е множеството от състояния достижими от , само чрез -преходи.

[redacted]

Множеството се пресмята лесно със следният алгоритъм:

$E(q) := \{q\}$

1. Първоначално

$E(q) := E(q) \cup \{r\}$

2. Докато има преход , такъв че и изпълни:

Този алгоритъм ще завърши след най-много броя на състоянията стъпки.

M'

Така вече можем да дефинираме формално автомата :

$K' = 2^K$

$s' = E(s)$

$F' = \{Q \subseteq K : Q \cap F \neq \emptyset\}$

$a \in \Sigma$

и за всяко и всяка буква :

$\delta(Q,a) = \tilde{} \{E(q) : \exists (q,a,p), p \in K, q \in Q\}$

$\emptyset \in K' = \emptyset$

Сега остава да докажем, че така получения автомат е детерминиран и еквивалентен на . Това, че е детерминиран се вижда лесно, тъй като функцията по дефиниция има единствена стойност във всяка една точка и е дефинирана за всяко едно състояние и всяка една буква от азбуката (това че е възможно , не означава че функцията не е дефинирана, тъй като).

Сега ако докажем твърдението

$(q,w) \bullet \overset{*}{}_M (p,e) \Leftrightarrow (E(q),w) \bullet \overset{*}{}_{M'} (P,e), p \in P$

$w \in L(M') \Leftrightarrow (s,w) \bullet \overset{*}{}_M (f,e), f \in F \Leftrightarrow (E(s),w) \bullet \overset{*}{}_{M'} (Q,e), f \in Q \Leftrightarrow (s',w) \bullet \overset{*}{}_{M'} (Q,e), Q \in F'$

ще докажем теоремата много лесно. Това е така, защото ако вземем , то, като последната част на твърдението е дефиницията за това .

$|w|$

Ще докажем горното твърдение по индукция върху .

$w = e$
$|w| = 0$

1. За , т.е. , трябва да покажем че

$(q,e) \bullet \overset{*}{}_M (p,e) \Leftrightarrow (E(q),e) \bullet \overset{*}{}_{M'} (P,e), p \in P$

$p \in E(q)$

Първото твърдение е еквивалентно на това да кажем че ,а второто твърдение на това че , т.е. , с което твърдението е доказано.

2. Предполагаме, че твърдението е вярно за всички думи с дължина до .

k

$a \in \Sigma$

3. Трябва да докажем, че твърдението е вярно за всички думи с дължина . Нека , където ,а .

$(E(q),w) \bullet \overset{*}{}_{M'}(R_1,a) \bullet {}_{M'}(P,e)$ (p,e)

a.) нека , тогава съществуват две състояния и , такива че , от където следва, че можем да кажем , но тъй като , от индукционното предположение следва, че . Тъй като , то съществува , следователно от дефиницията на следва, че . Тъй като , следва че , следователно . От тук следва че ,от където .

$(q,va) \bullet \overset{*}{}_{M}(r_1,a) \bullet {}_{M}(r_2,e) \bullet \overset{*}{}_{M}(p,e)$

b.) за да докажем твърдението в обратната посока да предположим че , за някое , което съдържа и някое , за което . От дефиницията на , знаем че е обединението от всички множества , където за някое , съществува . Тъй като , съществува някакво , такова че и за някое има преход . Следователно от дефиницията на , следва че . От индукционното предположение следва че , от където .

Така теоремата е доказана.

Σ^*

Дефиниция: Нека имаме крайна азбука . Множеството от регулярни изрази във е:
$a \in \Sigma$

1. и всяка буква е регулярен израз
,

$(\alpha\beta)$

2. Ако и са регулярни изрази, то също е регулярен израз
$(\alpha \cup \beta)$

3. Ако и са регулярни изрази, то също е регулярен израз
α^*

4. Ако е регулярен израз, то също е регулярен израз
5. Няма други регулярни изрази освен тези описани в точки 1 до 4

$\Lambda(\alpha)$

Дефиниция: Ако имаме даден регулярен израз , регулярен език е множеството, където е функция дефинирана по следния начин:
$a \in \Sigma = \{a\}$

1. 1. и за всяко .
$\Lambda((\alpha\beta)) = \Lambda(\alpha)\Lambda(\beta)$

2. Ако и са регулярни изрази, тогава
$\Lambda((\alpha \cup \beta)) = \Lambda(\alpha) \cup \Lambda(\beta)$

3. Ако и са регулярни изрази, тогава
$\Lambda(\alpha^*) = \Lambda(\alpha)^*$

4. Ако е регулярен израз, тогава

Теорема: Класа от езици, които се разпознават от крайни автомати е затворен относно операциите:

1. Обединение
2. Конкатенация
3. Звезда на Клини
4. Допълнение
5. Сечение

Доказателство(тук е добра идея да се приложат и малко диаграми):

За всеки един случай ще покажем как можем да построим автомат, който разпознава езика резултат от съответната операция.

$M = (K, \Sigma, \Delta, s, F)$ $L(M_2)$

А. Обединение: Нека имаме два недереминирани крайни автомата и . Ще построим недерминиран краен автомат, който за който . Това ще направим като направим едно ново състояние и сложим преходи от това състояние към началните състояния на двата автомата (можем да приложим абсолютно същата техника за да построим обединение на произволен брой автомати). Ето как би изглеждала дефиницията формално: нека новия автомат е като,

$K = K_1 \cup K_2 \cup \{s\}$

,

$F = F_1 \cup F_2$

,

$\Delta = \Delta_1 \cup \Delta_2 \cup \{(s, e, s_1), (s, e, s_2)\}$

$L(M) = L(M_1) \cup L(M_2)$

Така при всяко едно пускане недетерминистично решава, кой от двата езика да разпознава, като по този начин разпознава и двата езика. Формално казано: ако , тогава за някое , за някое или , за някое . Тогава разпознава разпознава или разпознава , т.е. .

M_2 $M) = L(M_1) L(M_2)$

Б. Слепване: Нека пак и са два недетерминирани крайни автомати. Тогава за да построим автомат , за който , ще обединим състоянията на двата автомата (без ограничение на общността можем да считаме ще състоянията на двата автомата са непресичащи се множества) и от всяко крайно състояние на ще направим преход към началното състояние на . Доказателството е аналогично на доказателството от предишната точка.

$M_1 M_1) = L(M)^*$

В. Звезда на Клини: Нека е недетерминиран краен автомат. Искаме да построим автомат , такъв че . За целта ще използваме идея подобна на тази, която използвахме при слепването. Новия автомат ще има същите

състояние и преходи като , но ще има ново начално състояние, което е и крайно, за можем да разпознаваме празната дума. Освен това от всяко едно крайно състояние ще направим преход към (началното състояние на), както и преход от към . Така когато прочетем дума от , можем да започнем от началното състояние на с нова дума.

$\overline{M} = (K, \Sigma, \delta, s, K - F)$

Г. Допълнение: Нека е краен детерминиран автомат. Тогава допълнението на езика на този автомат е се разпознава от детерминирания краен автомат , т.е. просто правим всяко не-крайно състояние крайно и всяко крайно, не-крайно.

Д. Сечение: Имайки доказателствата до тук можем просто да си припомним, че

$L_1 \cap L_2 = \Sigma^* - ((\Sigma^* - L_1) \cup (\Sigma^* - L_2))$

Забелязваме, че дясната страна на израза използва само операции, които вече описахме, с което доказателството е завършено.

Теорема(на Клини): Един език е регулярен, тогава и само тогава, когато се разпознава от краен автомат.

Доказателство:

$\varnothing$

) Очевидно е че крайните автомати разпознават , както всеки език съставен от една буква. Освен това от предната теорема видяхме, че езиците, които се разпознават от автомати са затворени относно операциите обединение, слепване и звезда на Клини. Следователно всеки един регулярен език се разпознава от някакъв краен автомат.

$k = n\ k)\ n\ q_n\}\ F)$

) Нека е краен автомат (не задължително детерминиран). Ще конструираме регулярен език , такъв че . За целта ще представим като обединение на няколко по-прости езика. Нека и . За и , ще дефинираме да е множеството от всички думи от , които могат да накарат да отиде от състояние до състояние , без да се преминава през състояния със номера по-големи от , като и е възможно да са по-големи от . Забелязваме че при :

$R(i,j,n) = \{w \in \Sigma^* : (q_i, w) \bullet \overset{*}{}_M (q_j, e)\}$

Следователно:

$L(M) = \sim \{R(1,j,n) : q_j \in F\}$

$L(M)\ k)$

Идеята е, да докажем че всяко от множествата е регулярно, от където ще следва че също е регулярен.
$i = j \cup \{a \in \Sigma \cup \{e\} : (q_i, a, q_j) \in \Delta\}$

Ще докажем това с индукция по . За , е или ако или ако . Всяко едно от тези множества е крайно, от където следва че е регулярно.
$R(i,j,k-1)$

Предполагаме, че за , всяко едно множество е регулярно.
$R(i,j,k)$

Трябва да докажем, че е регулярно. За целта ще представим това множество с операциите обединение, слепване и звезда на Клини по следния начин:
$R(i,j,k) = R(i,j,k-1) \cup R(i,k,k-1) R(k,k,k-1)^* R(k,j,k-1)$

k_j

Идеята тук е, че всеки път от до , които не минава през състояния с номер по-голям от е един от следните типове:
$k-1$

1. Минава през състояния с номера по-малки или равни на $k-1$

2. Минава от до , после се върти от до , нула или повече пъти и накрая отива от до , като всеки един от тези подпътища не минава през състояния с номер по-голям от .
$R(i,j,k)$

От тук следва, че е регулярен, което завършва доказателството.

$i \geq 0 \in L$

Лема на покачването: Нека е регулярен език. Тогава съществува цяло число , такова че всяка дума , такава че , може да бъде представена като , като , и , за всяко .

Доказателство(тук май може да се докаже и с картинка):
w

е регулярен, значи съществува краен автомат , които разпознава . Нека е броя състояния на и нека е дума с дължина или повече. Да предположим, че първите стъпки от смятането на за са:

$(q_0, w_1 w_2 \ldots w_n) \vdash_M (q_1, w_2 \ldots w_n) \vdash_M \ldots \vdash_M (q_n, e)$

n v $w_{j+1} \ldots w_m$ v_j

където е началното състояние на и са първите букви от . Тъй като има състояния, а минаваме през конфигурации, то съществува състояния и , , такива че . Това е поддумата , която води от състояние обратно в същото състояние, и това не е празната дума тъй като . Но тогава тази поддума може да бъде премахната от или повторена произволен брой пъти след -тата буква на и ще продължи да разпознава новополучената дума. Следователно ще разпознава , за всяко , където и . Накрая забелязваме, че дължината на е , което по предположението по-горе е най-много , така както се изисква в теоремата. С това теоремата е доказана.

Примери за регулярни и нерегулярни езици:

$xz = a^{n-i}b^n \notin L$

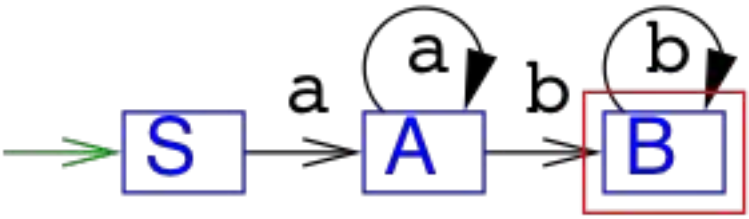
Езика не е регулярен. Ако беше тогава от горната теорема ще съществува , за което , такива че и , т.е. , но тогава , което противоречи на теоремата.

$p + sq + r = (q+1)(p+2q+r) > 0$

Езика не е регулярен. Ако беше тогава то горната теорема съществува , за което . Тогава . Тогава от теоремата следва че е просто, но за , получаваме , което е произведение на 2 естествени числа всяко от които по голямо от 1.

$L = \{a^n b^m : n > 0, m > 0\}$

Езика е регулярен. И неговия автомат би изглеждал така (автомата не е детерминиран, тъй като няма преход от S със b):



$\approx_L \in L \Leftrightarrow yz \in L$

Дефиниция: Нека е език и нека . Казваме че и са еквивалентни спрямо (), ако за всяко , следното е изпълнено: . Забележете, че е релация на еквивалентност.

$(s,y) \bullet \overset{*}{}_M (q,e)$

Дефиниция: Нека е детерминиран краен автомат. Казваме, че две думи , са еквивалентни спрямо (), ако има състояние , такова че и .

E_q

също е релация на еквивалентност и нейните класове на еквивалентност могат да бъдат идентифицирани, чрез състоянията на ,

които могат да бъдат достигнати от , чрез някаква дума. Ще означаваме класа на еквивалентност за дадено състояние , чрез .

$x \approx_{L(M)} y, \delta, s, F)$

Теорема: За всеки детерминиран краен автомат и две думи , ако , то .

$L(M)$

От горната теорема следва, че всеки един клас на еквивалентност спрямо се съдържа в клас на еквивалентност на и всеки един клас на еквивалентност на е обединение на един и няколко класа на еквивалентност на . Тъй като класовете на еквивалентност на са броя състояния на автомат разпознаващ , то можем да изкажем едно много важно свойство на всеки един автомат, които разпознава : всеки един автомат които разпознава трябва да има поне толкова състояния, колкото класове на еквивалентност има релацията . Следващата теорема доказва, че е възможно да се построи автомат разпознаващ с точно толкова състояния, което ще рече, че това е автомата с минимален брой състояния за езика .

$L(M)^*$

Теорема (на Михайл-Нероуд): Нека е регулярен език. Тогава съществува краен детерминиран автомат с брой състояния равен на броя на класове на еквивалентност на , който разпознава . Това е автомата с минимален брой състояния, който разпознава и този автомат е единствен с точност до изоморфизъм.

Доказателство:

$M = L(M), \Delta, \delta, F)$

Нека означим класа на еквивалентност спрямо , породен от , чрез . Ще построим краен детерминиран автомат, за който . Дефинираме по следния начин:

$\approx_L = \{[x] : x \in \Sigma^*\}$

, т.е. това са класовете на еквивалентност на

$s = [e]$

, класа на еквивалентност на празната дума

$F = \{[x] : x \in L\}$

$\delta([x], a) = [xa]$

Накрая дефинираме за всяко състояние и всяка буква , .

K_1

Така дефиниран има краен брой състояния, защото е регулярен, т.е. има краен детерминиран автомат , който го разпознава. От предишната теорема знаем, че има по-малко или равен брой класове на еквивалентност

в от колкото в , а има краен брой класове на еквивалентност тъй като има краен брой състояния, от където следва че и има краен брой класове на еквивалентност, т.е. е крайно.
M

Освен това е добре дефинирана функция на преходите, тъй като е еднозначна и дефинирана за всяко едно състояние на и всяка една буква от азбуката.
$x,y \in \Sigma^*)$

Остава да покажем, че . Първо ще покажем че за всеки имаме че:
$([x],y) \bullet \overset{*}{}_M ([xy],e)$

(1)

$([x],y'a) \bullet \overset{*}{}_M ([xy'],a) \bullet {}_M ([xy],e)$

Това се доказва с индукция по . За се доказва тривиално. Ако предположим, че е вярно за всяка една дължина на до и , тогава от индукционното предположение имаме: .
$x \in L^F M) \Leftrightarrow ([e],x) \bullet \overset{*}{}_M (q,e), q \in F$

Използвайки (1), доказателството е директно: за всяка , имаме че , което от (1) е същото като да кажем, че или от дефиницията на , .

$\approx_L$

Следствие: Един език е регулярен тогава и само тогава, когато релацията има краен брой класове на еквивалентност.

Алгоритъм за конструиране на минимален детерминиран автомат еквивалентен на даден детерминиран автомат:
$p \equiv_n q : (q,z) \in A_M \Leftrightarrow (p,z) \in A_M, z \in \Sigma^*, |z| \le n$

Нека е детерминиран краен автомат. Дефинираме релацията , като . Казваме, че 2 състояния са еквивалентни (означаваме), ако за . Класовете на еквивалентност на са тези множества от състояния, които дефинират състоянията на минималният автомат, който разпознава . За да намерим тези класове на еквивалентност, ще пресметнем последователно класовете на еквивалентност на , където . Това ще направим последователно като ще намираме класовете на еквивалентност на една релация от класовете на еквивалентност на предишната релация.
$K - F$

1. За е очевидно, че класовете на еквивалентност са и .

$\equiv_n$ $\forall a \in \Sigma, \delta(q,a) \equiv_n \delta(p,a)$

2. Да предположим, че сме намерили класовете на еквивалентност на . Тогава за всеки две състояния , и . По този начин можем да разберем дали 2 състояния са в един клас на еквивалентност или не от където можем да намерим класовете на еквивалентност на .
3. Продължаваме докато не получим, че класовете на $\equiv_{n-1}$ са същите като класовете на еквивалентност на еквивалентност на .

M

Алгоритъмът ще завърши след краен брой стъпки, тъй като на всяка една стъпка класовете на еквивалентност се увеличават поне с 1 и не може да има повече класове на еквивалентност от колкото състояния има в .